

Feb 23 HW 19

Q1: Thread vs Async

Multi-threading is best for CPU-bound tasks such as image processing or parallel computation.
Async/non-blocking I/O is best for I/O-bound tasks like web servers handling thousands of network requests.

Q2: Virtual Threads (Java 21)

Virtual threads are lightweight threads managed by the JVM (Project Loom).
Unlike platform threads (1:1 with OS threads), virtual threads allow millions of concurrent tasks.
Best for high-concurrency I/O workloads. Not ideal for heavy CPU-bound loops.

Q3: Lock vs CAS

Locks (synchronized, ReentrantLock) are blocking and easier for complex critical sections.
CAS (Compare-And-Swap) is non-blocking and used in Atomic classes.
Use locks for complex logic; use CAS for high-performance atomic counters.

Q4: Deadlock Conditions

Four necessary conditions:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Lock ordering prevents deadlock by enforcing a global lock acquisition order.

Q5: Thread Pool Sizing

Formula:

$\text{Threads} = \text{CPU cores} \times (1 + \text{WaitTime} / \text{ComputeTime})$

CPU-bound on 8 cores $\rightarrow$ 8 threads.

I/O-bound (100ms wait, 5ms compute) $\rightarrow 8 \times (1 + 100/5) = 168$ threads.

Q6: Exponential Backoff with Jitter

Exponential backoff increases delay as $\text{base} \times 2^{\text{attempt}}$.
Jitter adds randomness to prevent retry storms and thundering herd problems.

Q7: Circuit Breaker Pattern

States:

CLOSED $\rightarrow$ normal operation
OPEN $\rightarrow$ reject requests after failures exceed threshold
HALF-OPEN $\rightarrow$ test requests allowed

Transitions:

CLOSED $\rightarrow$ OPEN (too many failures)
OPEN $\rightarrow$ HALF-OPEN (after timeout)
HALF-OPEN $\rightarrow$ CLOSED (success)
HALF-OPEN $\rightarrow$ OPEN (failure)

Q8: Exactly-Once Semantics

Exactly-once delivery is impossible due to network failures and message duplication.
Instead, implement exactly-once processing using idempotency keys and deduplication logic.

Q9: Caching Patterns

Cache-Aside: App loads data into cache on miss. Simple but eventual consistency.
Write-Through: Write to cache and DB together. Strong consistency but slower writes.
Write-Behind: Write to cache first, DB later. Fast but risk of data loss.

Q10: Cache Stampede

Cache stampede happens when many requests hit DB after cache expiry.

Solutions:

1. Mutex/locking during cache rebuild.
2. Randomized TTL or early refresh to avoid synchronized expiration.